1. Test execution summary

The test cases were automated with pytest and pytest-django and executed against a fresh test database created for the run.

Item	Value
Command	pytest tests/test_lab4_blackbox.py tests/test_lab4_whitebox.py -v
Environment	Windows 11, Python 3.13.6, Django 5.2.7, pytest 8.4.2, pytest-django 4.11.1
Duration	0.94 s for the 44 test cases

All test cases share the same data unless stated otherwise, every test uses the five flats below, with each flat's nearest bus stop and MRT exit being 80m and 500m away respectively.

The tests are built around a **reference search** (a realistic search that a user could enter: budget $350,000 – $700,000, 4 ROOM, remaining lease of at least 80 years). Flats A - E are modified entries to test the 4 filters that exist within the filter_flats() function. The success criterion is that the reference search only returns flat A, and that leaving any one filter blank adds exactly the one flat that filter was blocking.

This setup lets the tests expose bugs in the filter code:

- Run the reference search as it is. Only A should be returned. If B, C, D or E appears, the filter that should have blocked it is not working.
- Leave one filter blank at a time. Exactly one more flat should appear. If it doesn't, that filter is still being applied when it should be skipped.

Flat	Flat type	Resale price	Remaining lease	Result of the reference search
Reference Search	***4 ROOM***	***$350,000 - $700,000***	***>= 80 years***	**-**
A	4 ROOM	$600,000	90 years	✔ Included (matches every filter)
B	4 ROOM	**$250,000**	85 years	✘ Excluded: below minimum budget
C	4 ROOM	**$900,000**	95 years	✘ Excluded: above maximum budget
D	**5 ROOM**	$500,000	85 years	✘ Excluded: wrong flat type
E	4 ROOM	$400,000	**70 years**	✘ Excluded: remaining lease too short

2. Black box testing – SearchController

The SearchController control class covers four use cases: UC-01 Search, UC-08 Filter, UC-09 Rank and UC-10 Sort. To test it, each input was divided into equivalence classes:

a. **Range inputs** (e.g. budget, lease, walking distance) have one valid class and two invalid classes: below the range and above it.
b. **Discrete inputs** (e.g. flat type) have one valid class (a value from the allowed set) and one invalid class (any other value).

Boundary values were tested exactly on each boundary, which should be accepted, and one step outside it, which should be rejected. Values just inside a boundary belong to the same class as the boundary itself, so they were not tested separately.

Invalid inputs never reach the controller, because the SearchForm boundary object rejects them first. Those test cases therefore run through the full chain:

SearchForm → SearchQuery → SearchController

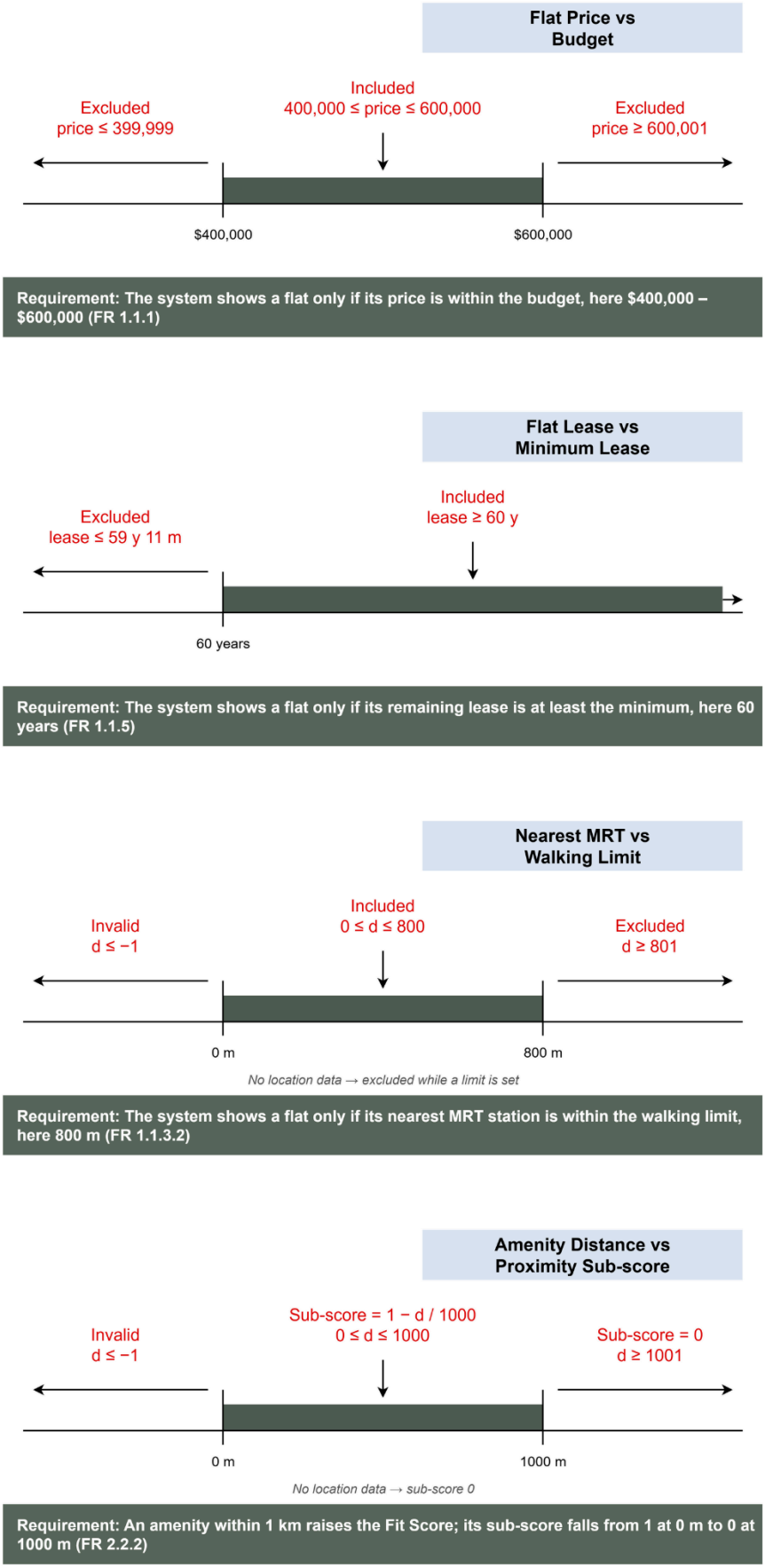
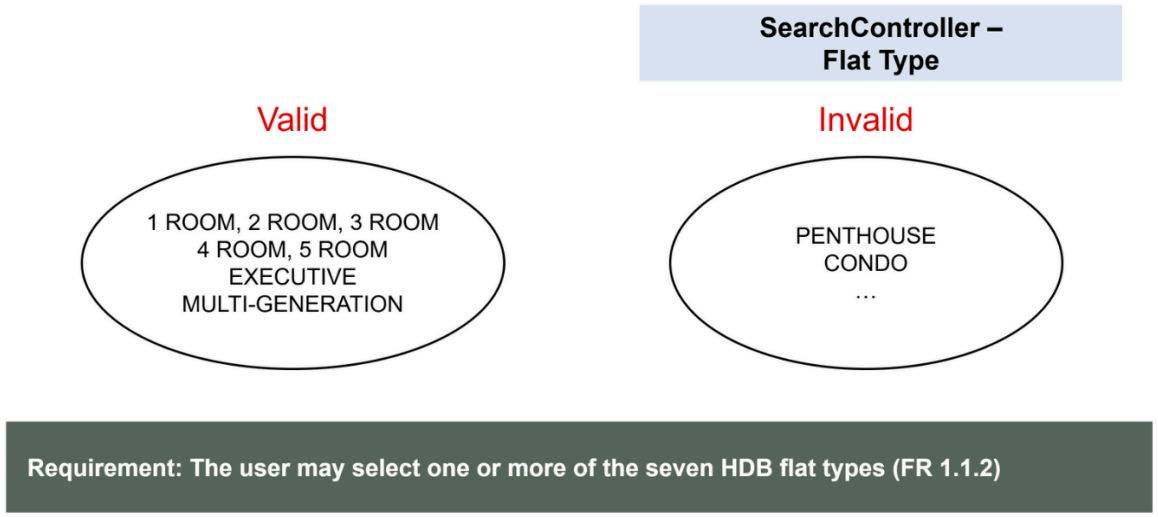


Figure 1 – Equivalence classes and boundary values of the range parameters





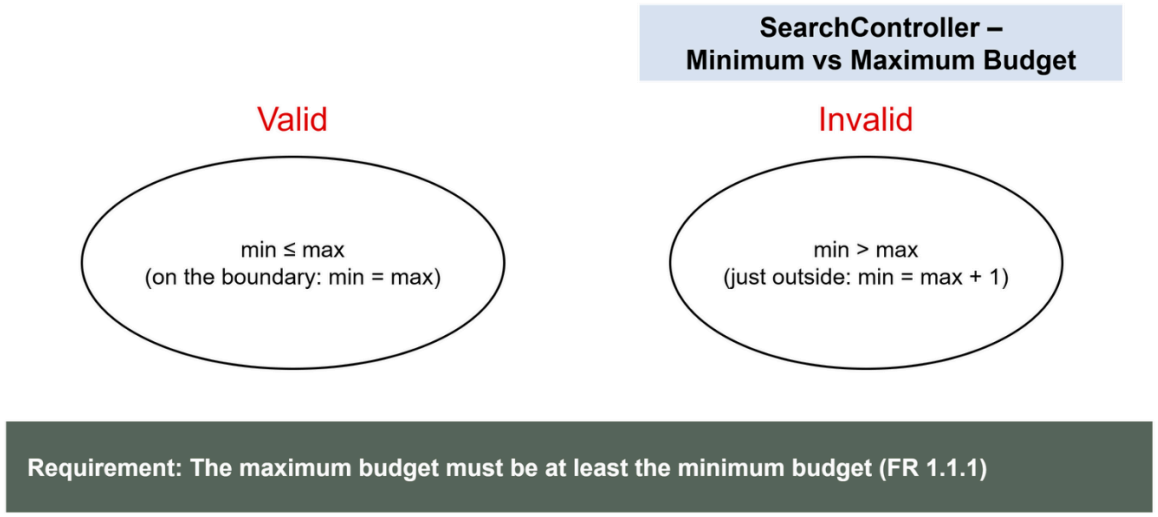


Figure 2 – Equivalence classes of the discrete parameters

2.1 Valid inputs (several valid inputs combined per case)

ID	Test input	Expected output	Actual output	Result
BB-V1	**Budget**: $0 - $10,000,000 **Flat Type**: 4 ROOM **Min Lease**: 0 yrs **Walk Limit**: Bus Stop 100m, MRT exit 2000m. **Weights**: all 1 **Sort by**: Price, ascending	B, E, A, C (D: wrong flat type)	B, E, A, C	**PASS**
BB-V2	**Budget**: $10,000,000 - $10,000,000 **Flat Type**: all 7 types **Min Lease**: 99 yrs **Walk Limit**: Bus Stop 2000m, MRT exit 100m. **Weights**: all 10 **Sort by**: Fit Score, descending	No Results	No Results	**PASS**
BB-V3	**Budget**: $0 - $0 **Flat Type**: EXECUTIVE **Min Lease**: None **Walk Limit**: Bus Stop: Any, MRT Exit: Any **Weights**: *ignore all* **Sort by**: Floor area, descending	No Results	No Results	**PASS**
BB-V4	(no parameters)	All five flats (A–E)	A, B, C, D, E	**PASS**

2.2 Invalid inputs (one invalid input per case)

While one parameter is tested, all other parameters keep the valid baseline values: `min_budget=300000, max_budget=700000, flat_type=4 ROOM, min_lease=60, max_walk_bus=400, max_walk_mrt=800, w_*=5, sort=fit_score-desc`.

ID	Test input	Expected output	Actual output	Result
BB-I1	min_budget=-1	Error on min_budget only; no search run	min_budget: "Ensure this value is greater than or equal to 0." No search run.	**PASS**
BB-I2	min_budget=10000001	Error on min_budget only; no search run	min_budget: "Ensure this value is less than or equal to 10000000." No search run.	**PASS**
BB-I3	max_budget=-1	Error on max_budget only; no search run	max_budget: "Ensure this value is greater than or equal to 0." No search run.	**PASS**
BB-I4	max_budget=10000001	Error on max_budget only; no search run	max_budget: "Ensure this value is less than or equal to 10000000." No search run.	**PASS**

BB-I5	min_budget=700001 (max 700000)	Error on max_budget only: "Maximum budget must be at least the minimum budget." No search run.	max_budget: "Maximum budget must be at least the minimum budget." No search run.	**PASS**
BB-I6	min_lease=-1	Error on min_lease only; no search run	min_lease: "Ensure this value is greater than or equal to 0." No search run.	**PASS**
BB-I7	min_lease=100	Error on min_lease only; no search run	min_lease: "Ensure this value is less than or equal to 99." No search run.	**PASS**
BB-I8	max_walk_bus=99	Error on max_walk_bus only; no search run	max_walk_bus: "Ensure this value is greater than or equal to 100." No search run.	**PASS**
BB-I9	max_walk_bus=2001	Error on max_walk_bus only; no search run	max_walk_bus: "Ensure this value is less than or equal to 2000." No search run.	**PASS**
BB-I10	max_walk_mrt=99	Error on max_walk_mrt only; no search run	max_walk_mrt: "Ensure this value is greater than or equal to 100." No search run.	**PASS**
BB-I11	max_walk_mrt=2001	Error on max_walk_mrt only; no search run	max_walk_mrt: "Ensure this value is less than or equal to 2000." No search run.	**PASS**
BB-I12	w_mrt_station=0	Error on w_mrt_station only; no search run	w_mrt_station: "Ensure this value is greater than or equal to 1." No search run.	**PASS**
BB-I13	w_mrt_station=11	Error on w_mrt_station only; no search run	w_mrt_station: "Ensure this value is less than or equal to 10." No search run.	**PASS**
BB-I14	flat_type=PENTHOUSE	Error on flat_type only; no search run	flat_type: "Select a valid choice. PENTHOUSE is not one of the available choices." No search run.	**PASS**
BB-I15	sort=rating-asc	Not rejected; sort falls back to Fit Score, descending; results A, E	Form valid; sorted by Fit Score, descending; results A (34.8), E (34.7)	**PASS**

2.3 Filter boundaries (flat attribute vs. criterion)

Each case creates one flat on, or just outside, the edge of a criterion and checks whether filter_flats() keeps it.

Budget = $400,000 - $600,000

Minimum lease 60 years (smallest unit = 1 month)

MRT walk limited to 800 m.

ID	Test input	Expected output	Actual output	Result
BB-F1	price $399,999	Excluded	Excluded	**PASS**
BB-F2	price $400,000	Included	Included	**PASS**
BB-F3	price $600,000	Included	Included	**PASS**
BB-F4	price $600,001	Excluded	Excluded	**PASS**
BB-F5	lease 59 y 11 m (59.917)	Excluded	Excluded	**PASS**
BB-F6	lease 60 y 0 m	Included	Included	**PASS**
BB-F7	nearest MRT 800 m	Included	Included	**PASS**
BB-F8	nearest MRT 801 m	Excluded	Excluded	**PASS**
BB-F9	no coordinates / no proximity data	Excluded while a limit is set	Excluded	**PASS**
BB-F10	Nearest MRT -1m	Included (not validated)	Included	**PASS**

2.4 Fit Score MRT sub-score (1 km radius, mrtStationWeight = 10)

ID	Test input	Expected output	Actual output	Result
BB-S1	nearest MRT 0 m	mrtStationScore = 1.0	1.0	**PASS**
BB-S2	nearest MRT 250 m	mrtStationScore = 0.75	0.75	**PASS**
BB-S3	nearest MRT 1000 m	mrtStationScore = 0.0	0.0	**PASS**
BB-S4	nearest MRT 1001 m	mrtStationScore = 0.0 (clamped)	0.0	**PASS**
BB-S5	no coordinates	mrtStationScore = 0.0	0.0	**PASS**
BB-S6	Nearest MRT -1m	mrtStationScore = 1.0	1.0	**PASS**

3. White box testing – SearchController.filter_flats() (§3.2.2)

Lines 52-63 in `backend/core/services/search.py` show 4 independent if statements to apply the budget, flat type and lease filters before apply_walk_limits().

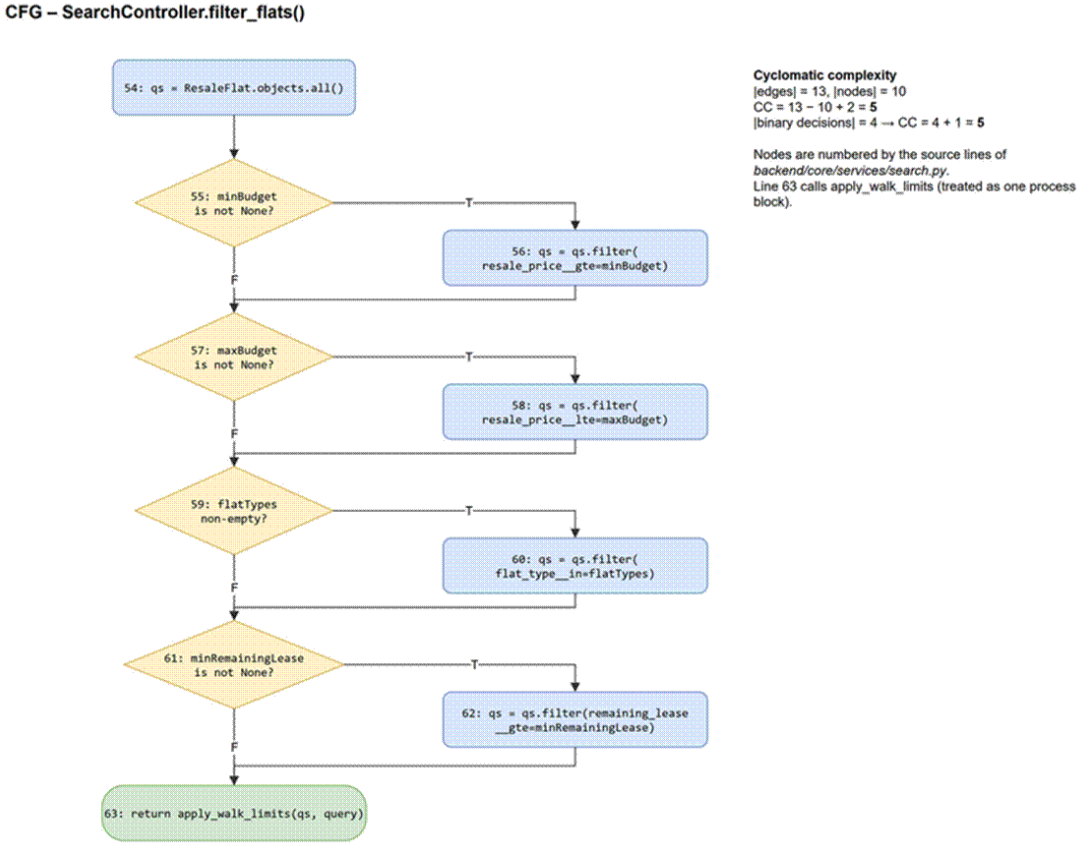


Figure 3 – Control flow graph of filter_flats()

Cyclomatic complexity: |edges| = 13, |nodes| = 10, so CC = 13 − 10 + 2 = 5 (check: 4 binary decisions + 1 = 5). Five basis paths are needed. The baseline is the typical full search (every criterion set). Each further path flips one decision and keeps the rest as in the baseline. As the four if statements are independent, all five paths are feasible.

ID	Basis path	Test input (SearchQuery)	Expected	Actual	Result
WB-F1	`54,55,56,57,58,59,60,61,62,63`	min $350k, max $700k, [4 ROOM], lease ≥ 80	{A}	{A}	**PASS**
WB-F2	`54,55,57,58,59,60,61,62,63`	min blank, max $700k, [4 ROOM], lease ≥ 80	{A, B}	{A, B}	**PASS**
WB-F3	`54,55,56,57,59,60,61,62,63`	min $350k, max blank, [4 ROOM], lease ≥ 80	{A, C}	{A, C}	**PASS**
WB-F4	`54,55,56,57,58,59,61,62,63`	min $350k, max $700k, types blank, lease ≥ 80	{A, D}	{A, D}	**PASS**
WB-F5	`54,55,56,57,58,59,60,61,63`	min $350k, max $700k, [4 ROOM], lease blank	{A, E}	{A, E}	**PASS**

The five paths execute all 10 nodes and all 13 edges: 100% statement and branch coverage.

4. White box testing – parse_weights()

Lines 89–101 in `backend/core/services/compare.py` read the Compare page's weight sliders, with a for loop, an if/elif and a try/except.

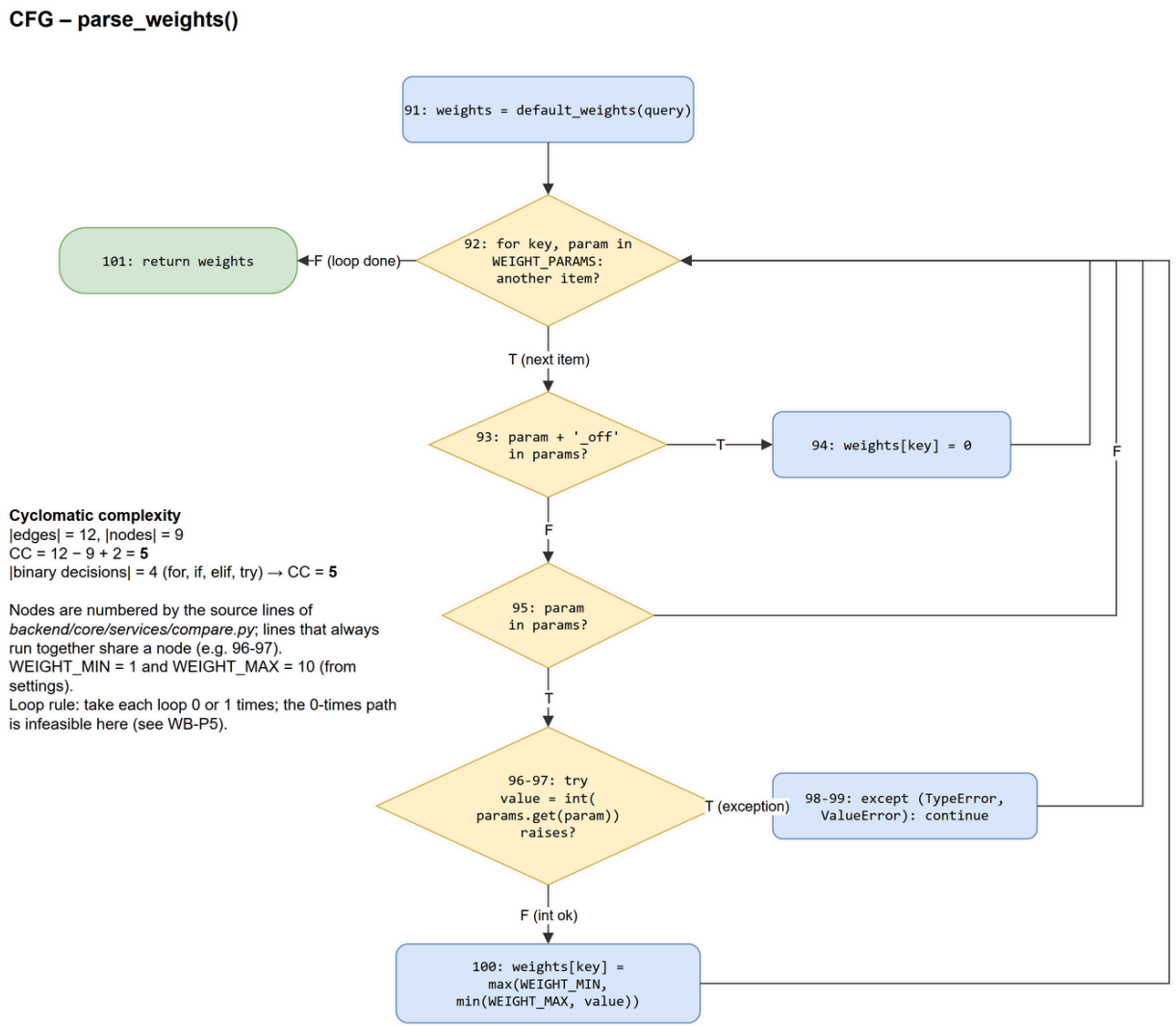


Figure 4 – Control flow graph of parse_weights()

Cyclomatic complexity: |edges| = 12, |nodes| = 9, so CC = 12 − 9 + 2 = 5. As a check, there are 4 binary decisions (for, if, elif, try), so CC = 4 + 1 = 5. We treat the try block as one more binary decision. int() either raises an exception (go to 98-99) or it does not (go to 100). Lines that run in sequence are merged into one node.

Normally, the baseline would skip the loop. However, the path (91, 92, 101) is infeasible here because WEIGHT_PARAMS is a fixed, non-empty constant, so the loop always runs. To minimize the number of infeasible paths, we chose a feasible baseline that enters the loop

once. Each remaining path flips one decision and keeps the rest as in the baseline. The path that skips the loop is still listed as WB-P5 and marked infeasible.

ID	Basis path	Test input (params)	Expected	Actual	Result
WB-P1	`91,92,93,95,96-97,100,92,101`	cw_price=8	price = 8; others default	price = 8; others default	**PASS**
WB-P2	`91,92,93,94,92,101`	cw_price_off=on	price = 0; others default	price = 0; others default	**PASS**
WB-P3	`91,92,93,95,92,101`	(empty)	exactly the defaults	exactly the defaults	**PASS**
WB-P4	`91,92,93,95,96-97,98-99,92,101`	cw_price=abc	no exception; price stays 5	no exception; price 5	**PASS**
WB-P5	`91,92,101`	—	Infeasible – not executable	—	**N/A**

The four feasible paths cover all 9 nodes and all 12 edges (edge 92→101 is taken whenever the loop ends): 100% statement and branch coverage.

3.3.2 SearchController Class Regeneration

For this section, we used Claude (Opus 5.5) as the AI model. We gave it a copy of our GitHub repository after deleting backend/core/services/search.py, so that it could read the necessary files and run tests. Afterwards, we prompted it to regenerate the SearchController class from scratch, and run pytest to fix failures until the tests passed. Its first attempt at regenerating the class was able to pass all 42 test cases and 275 tests in the suite, so no further prompting was needed. Upon comparing it with our original, we discovered some differences that were not detected by the tests. It scored the flats in a plain Python loop instead of using numpy, so it took 0.29s to rank 25,000 flats, whereas ours took 0.12s. This is still within NFR 1, but about 2.5 times slower. It also scored flats that lack coordinates as zero, whereas our code still used their stored distances.

The AI assistant's output met our expectations. Our README and CROSS_CHECK had the formulas and signatures written out explicitly, which likely played a significant role in helping it to succeed at regenerating the class on its first attempt. However, one key takeaway we had was that passing all of the tests did not mean that the class was exactly the same, so our test suite needs to be edited to include a case for flats without coordinates, as well as a performance check.

3.3.3 AI Codebase Quality Reflection

For this section, we chose to ask for the AI's opinion on the quality of our codebase. Using the same AI model on the same repository, we prompted it to review the code quality of our codebase and suggest refactorings as necessary. We also prompted it to ensure that all existing tests must still pass. The AI model reported six issues, as listed:

1. The Compare page crashes on any flat without a proximity record. It reads the record in a way that raises an exception when the record is missing, whereas the Fit Score handles this case safely.
2. The values 99, 1 and 10 are hard-coded instead of coming from settings.
3. The rule for choosing a block's best flat is duplicated in two views.
4. The Compare score re-implements the Fit Score formula.
5. flat_details() is overly long.
6. The code mixes camelCase and snake_case names.

Most importantly, it reported finding a bug in our codebase. In our search page, the code starts by checking whether or not the flat listing has a distance record, and if there is not one, it sets the distances to zero. In our Compare page, however, the code does not check for whether or not the flat listing has a distance record, so it could cause the page to crash if there was not a distance record. The AI model suggested fixing it by making the Compare page also treat missing distance records as zero.

Overall, the AI model met our expectations and helped us to find a real inconsistency that we had missed. However, the impact of the bug would likely have been very small because a fully loaded database gives every flat a proximity record.